NAME: Jeevitha.S
PROJECT TOPIC: Food Delivery Management System

Assignment 4: Relational Database vs. NoSQL Database

DBMS Coursework | Comparative Paradigm Analysis for Food Delivery Management Systems

1. Comparative Paradigm Overview

Modern architectures often require choosing between structured Relational Database Management Systems (RDBMS) and flexible NoSQL systems. In the context of a **Food Delivery Management System**, each technology offers distinct trade-offs across storage, consistency models, and execution characteristics.

Dimension	Relational Database (RDBMS - e.g., MySQL)	NoSQL Database (e.g., MongoDB, Redis)
Data Model	Structured tables with pre-defined schema, rows, and columns. Enforces foreign key relationships.	Dynamic, schema-less structures (Document JSON, Key-Value, Columnar, or Graph).
Scalability	**Vertical Scaling (Scale-Up):** Increases CPU/RAM on single node. Horizontal sharding is complex.	**Horizontal Scaling (Scale-Out):** Seamless distribution of data across commodity clusters via auto-sharding.
Consistency Model	**ACID Guarantees:** Strong consistency across multi-table transactions (Atomicity, Consistency, Isolation, Durability).	**BASE Model:** Basically Available, Soft-state, Eventual consistency. Prioritizes high availability.
Performance Profile	Optimized for complex joins, multi-table transactional logic, and relational consistency queries.	Optimized for ultra-fast high-throughput reads/writes, unstructured key-value lookups, and real-time streaming.
Query Language	Standard SQL (Structured Query Language) supporting complex aggregations and joins.	Unstructured/JSON-based query APIs (e.g., MongoDB Aggregation Framework, Redis Commands).

Primary Strengths	Zero data loss, strict integrity constraints, structured schema enforcement, transactional safety.	High write availability, flexible schema modification, low-latency key lookups, horizontal elasticity.

2. Operational Dimension Analysis

Scalability Requirements

RDBMS: Handles growing transactional volumes by upgrading server hardware. While read-replicas scale reads well, multi-node writes with strict ACID properties require complex sharding middleware.
NoSQL: Designed natively for distributed execution. Adding nodes horizontally allows an engine like MongoDB or Cassandra to handle massive scale spikes effortlessly during peak dinner hours.

Consistency vs. Availability (CAP Theorem)

RDBMS (CP/CA): Ensures absolute financial and transactional correctness. A payment and order status update is committed synchronously, preventing double charging or orphaned items.
NoSQL (AP): Favors availability and partition tolerance. Allows delivery driver GPS tracking writes to succeed even if full cluster synchronicity is delayed by a few milliseconds.

3. Application in Food Delivery Systems

A modern enterprise food delivery platform (such as Uber Eats or Swiggy) rarely relies on a single database technology. Instead, a **Polyglot Persistence Architecture** is deployed to match specific platform sub-modules with their ideal database paradigm.

Sub-Systems Suited for RDBMS

• **Order Processing & Payments:** Strict ACID transactions guarantee that payment deductions, order record creation, and restaurant menu stock reductions succeed together or roll back entirely.
• **Financial Settlement & Billing:** Auditing partner earnings and customer receipts demands structured integrity, strict foreign keys, and zero anomaly risk.
• **User Profile & Role Management:** Well-defined schemas for Customers, Restaurants, and Delivery Partners with relational constraints.

Sub-Systems Suited for NoSQL

• **Live GPS Tracking (Redis / MongoDB):** Storing driver latitude/longitude updates every 3 seconds requires massive write throughput without transaction locks.
• **Real-Time Session Caching (Redis):** Caching active shopping carts, user sessions, and nearby restaurant lookup results in RAM for sub-millisecond response times.

• **Unstructured Catalog Search (Elasticsearch/MongoDB):** Handling highly variable restaurant menus, customization tags, and full-text item searches.

4. Organizational Decision Matrix: Why Choose Which Technology?

Key Decision Factors for Platform Engineering Teams

Organizations evaluate four critical criteria when selecting database engines for new service modules:

1. **Transaction Integrity Needs:** If data corruption or partial writes risk financial loss (e.g., payments, orders), **RDBMS** is mandatory.
2. **Schema Stability & Evolution:** Standardized entity schemas suit **RDBMS**. Frequently changing product catalogs or unstructured user activity logs benefit from **NoSQL**.
3. **Latency & Throughput Profiles:** Sub-millisecond caching and telemetry ingestion require **NoSQL key-value/document stores**.
4. **Operational Complexity & Cost:** Maintaining small-to-medium datasets on single RDBMS nodes is cost-effective, while global horizontal scalability demands NoSQL distributed clusters.

5. Comparative Summary & Recommendations

Use Case Module	Recommended DB	Core Justification
Core Order & Payment Ledger	RDBMS (MySQL / PostgreSQL)	Requires strict ACID properties, foreign key constraints, and relational consistency.
Live Driver Location Streams	NoSQL (Redis Geospatial / MongoDB)	Handles high-frequency write operations with minimal latency and flexible spatial indexing.
Dynamic Menu Recommendations	NoSQL (Document Store / Graph)	Flexible JSON schemas handle complex item variants, add-ons, and dynamic tag relationships.